



Université Mohammed Premier Oujda
École Nationale des Sciences Appliquées d'Oujda (ENSAO)

An Agentic MCP-Powered AI Ecosystem for Data Analytics

*From Dataset Ingestion to Automated Dashboard Publishing via
Collaborative Multi-Agent Orchestration*

Projet de Fin d'Année (2025–2026)

Réalisé par :

Mlle. SAHLI Oumaima
Mlle. AZIZI Safae
M. ELKHARRATI Jawad
M. GUINI Youssef

Encadré par :

M. ABDELMOUNAIM KERKRI

Membres du jury :

Pr. ABDELMOUNAIM KERKRI — Encadrant (ENSAO)
Pr. Mezouari — Membre de jury

Année universitaire : 2025–2026

Rapport réalisé dans le cadre du Projet de Fin d'Année à l'ENSA Oujda

REMERCIEMENTS

En préambule à ce rapport, qui marque une étape cruciale de notre formation d'ingénieurs au sein du module IDSCC4, nous tenons à exprimer notre profonde gratitude et notre respectueux dévouement à toutes les personnes qui ont contribué, de près ou de loin, à la réussite de ce projet.

Nous adressons nos remerciements les plus sincères à notre encadrant, **Monsieur Abdelmounaim Kerkri**. Nous le remercions chaleureusement pour la confiance qu'il nous a accordée en nous confiant ce sujet novateur, ainsi que pour sa disponibilité constante, sa patience et la pertinence de ses orientations scientifiques et techniques. Ses critiques constructives, sa rigueur méthodologique et ses conseils avisés ont été de précieux guides tout au long de la réalisation de ce travail, nous poussant continuellement à dépasser nos limites.

Nous lui sommes particulièrement reconnaissants de nous avoir guidés à travers les paradigmes complexes et innovants des architectures multi-agents et du protocole MCP (*Model Context Protocol*). Cette immersion technologique a considérablement enrichi notre bagage technique et a ouvert de nouvelles perspectives pour notre future carrière en Data Science et Cloud Computing.

Nos vifs remerciements s'adressent également à l'ensemble des membres du jury qui ont accepté d'évaluer notre travail et de nous faire l'honneur d'examiner ce rapport. Leurs remarques et suggestions seront assurément d'une grande valeur pour perfectionner nos compétences.

Nous tenons à exprimer notre reconnaissance à l'ensemble du corps professoral et administratif de l'**École Nationale des Sciences Appliquées d'Oujda (ENSAO)**. Nous saluons la haute qualité des enseignements qui nous sont dispensés et le dévouement des professeurs qui nous dotent, jour après jour, des outils conceptuels et pratiques indispensables pour relever les défis complexes du monde industriel et technologique moderne.

Une pensée particulière s'adresse à nos familles respectives. Nous ne saurions exprimer l'immensité de notre gratitude pour leur soutien inconditionnel, leurs sacrifices, leurs encouragements constants et leur affection tout au long de nos études. Qu'ils trouvent dans ce modeste travail le témoignage de notre profonde reconnaissance.

Enfin, nous tenons à saluer la synergie, l'engagement indéfectible et l'esprit d'équipe dont chacun des membres de ce groupe a fait preuve. C'est grâce à cette collaboration harmonieuse, au respect mutuel et à une persévérance partagée que ce projet a pu surmonter les obstacles techniques pour aboutir dans les meilleures conditions.

RÉSUMÉ

Ce projet vise à concevoir un écosystème intelligent et automatisé d'agents IA collaboratifs, capable de transformer un jeu de données brut en un dashboard interactif et un rapport d'analyse, sans intervention humaine à chaque étape. L'objectif principal est de démontrer la faisabilité d'une architecture multi-agent orchestrée autour d'un serveur MCP (*Model Context Protocol*), où chaque agent IA dispose d'un rôle, d'un ensemble d'outils autorisés, et collabore avec les autres agents pour accomplir un pipeline complet d'analyse de données.

Pour cela, nous avons développé six agents spécialisés : Orchestrateur, Data Engineer, Data Scientist, BI/Visualisation, DevOps et Reporter chacun reposant sur un modèle de langage (Llama 3.1 via l'API Groq) capable de raisonner et de décider quels outils appeler via un mécanisme de *tool-use*. Ces agents communiquent exclusivement à travers un serveur MCP central qui contrôle les permissions, exécute les outils et journalise chaque appel pour assurer la traçabilité du pipeline.

Le système repose sur une architecture modulaire intégrant un pipeline de traitement complet : ingestion et profilage du jeu de données, nettoyage et détection adaptative du domaine métier, calcul des indicateurs de performance (KPIs), génération de graphiques interactifs, publication d'un dashboard, puis compilation d'un rapport final. Le système a été validé sur le jeu de données réel *Online Retail II* (UCI Machine Learning Repository), contenant plus d'un million de transactions e-commerce, avec des résultats concrets : un score de qualité des données de 0,954 sur 1, un chiffre d'affaires total calculé de 20 972 594,55 £, et une détection automatique du domaine métier par le modèle de langage.

Cette solution démontre la faisabilité d'un système d'analyse de données entièrement automatisé par agents IA, combinant orchestration intelligente, contrôle d'accès via le protocole MCP, et gratuité d'usage grâce à l'utilisation de modèles open-source via Groq. Elle réduit significativement l'intervention humaine nécessaire pour produire des analyses business exploitables à partir de données brutes.

Table des matières

Remerciements	1
Résumé	2
1 Introduction Générale	7
1.1 Contexte et motivation	7
1.2 Problématique	7
1.3 Objectifs du projet	8
2 Fondamentaux : Agents IA et Protocole MCP	9
2.1 Introduction aux agents IA	9
2.1.1 Principe du <i>tool-use</i>	9
2.2 Le protocole MCP (Model Context Protocol)	10
2.2.1 Pourquoi un serveur MCP plutôt qu'un appel direct ?	10
2.3 L'orchestration multi-agent	11
2.4 Choix du modèle de langage : Llama 3.1 via Groq	11
3 Méthodologie et Conception du Système	13
3.1 Choix des outils et technologies	13
3.1.1 Langages et frameworks	13
3.1.2 Librairies utilisées	14
3.1.3 Environnement de développement	14
3.2 Architecture globale du système	14
3.2.1 Les neuf outils MCP	26
3.2.2 Table des permissions	27
3.3 Implémentation des agents	27
3.3.1 La classe BaseAgent	28
3.3.2 Le serveur MCP	28
3.3.3 Orchestration du pipeline (Engine)	29
3.4 Jeu de données et préparation des données	29
3.4.1 Présentation du dataset	29
3.4.2 Problèmes de qualité identifiés	30
3.4.3 Stratégie de nettoyage adaptative	30
3.4.4 Détection adaptative du domaine métier	30
4 Pipeline et Résultats Obtenus	32
4.1 Déroulement du pipeline complet	32
4.2 Résultats du Data Engineer	32
4.3 Résultats du Data Scientist	33
4.3.1 Top 5 pays par chiffre d'affaires	33

4.3.2	Chart hints transmis au BI Agent	33
4.4	Tests de validation du serveur MCP	34
4.5	Artefacts produits par un run complet	34
5	Interface et Dashboard	35
5.1	Présentation de l'interface développée	35
5.2	Suivi en temps réel de l'activité des agents	36
5.3	Dashboard interactif de visualisation	36
5.4	Consultation du rapport final	38
5.5	Bilan de l'interface	38
6	Démonstration et Conclusion	39
6.1	Démonstration de l'application	39
6.1.1	Lancement du pipeline	39
6.1.2	Exécution séquentielle des agents	39
6.2	Limites identifiées	41
6.3	Perspectives d'évolution	42
6.4	Conclusion	42
	Bibliographie	43

Table des figures

2.1	Boucle de décision d'un agent IA avec tool-use	10
2.2	Architecture du serveur MCP et de ses interactions avec les agents	11
3.1	Échanges entre le serveur MCP, les agents IA et l'interface utilisateur	13
3.2	Architecture globale du système multi-agent	15
3.3	Pipeline du Data Scientist Agent : gestion des colonnes anonymes vs non-anonymes	19
3.4	Boucle de décision dans BaseAgent (extrait)	28
3.5	Route principale du serveur MCP (extrait)	28
3.6	Route principale du serveur MCP (suite)	29
3.7	Boucle principale de l'Engine (extrait)	29
4.1	Pipeline complet du système, de l'ingestion à la publication du rapport . . .	32
5.1	Interface principale de soumission du jeu de données	35
5.2	Suivi en temps réel de l'activité des agents	36
5.3	Tuiles de KPIs synthétiques du dashboard	37
5.4	Graphiques interactifs du dashboard généré par le BI Agent	38
5.5	rapport final	38
6.1	Démonstration du système MCP et du pipeline	39
6.2	Extrait du journal d'exécution d'un run complet	40
6.3	Dashboard final consultable après exécution complète du pipeline	41

Liste des tableaux

3.1	Rôles et responsabilités des agents du système	26
3.2	Liste des outils enregistrés sur le serveur MCP	27
3.3	Table des permissions par agent	27
3.4	Caractéristiques du jeu de données Online Retail II	30
3.5	Problèmes de qualité détectés dans le dataset brut	30
4.1	Résultats du nettoyage par le Data Engineer	33
4.2	KPIs calculés sur le jeu de données Online Retail II	33
4.3	Répartition du chiffre d'affaires par pays (top 5)	33
4.4	Chart hints générés par le Data Scientist	33
4.5	Résultats des tests de validation du serveur MCP	34

Chapitre 1

Introduction Générale

1.1 Contexte et motivation

Avec l’explosion du volume de données générées quotidiennement par les entreprises transactions commerciales, interactions clients, journaux applicatifs l’analyse de données est devenue une composante stratégique incontournable pour la prise de décision. Cependant, le processus classique d’analyse de données reste largement manuel : un analyste doit charger les données, les nettoyer, choisir les indicateurs pertinents, produire des visualisations et rédiger un rapport. Ce processus est chronophage, répétitif, et dépend fortement de l’expertise humaine disponible.

Parallèlement, les avancées récentes des grands modèles de langage (LLM) et de leur capacité à appeler des outils externes (*tool-use* ou *function calling*) ouvrent la voie à une nouvelle génération de systèmes : des agents IA autonomes capables de raisonner, de décider quelles actions entreprendre, et de collaborer entre eux pour accomplir des tâches complexes. Le protocole MCP (*Model Context Protocol*), récemment standardisé, permet justement de structurer cette collaboration en définissant un cadre commun par lequel les agents accèdent à des outils partagés, de manière contrôlée et traçable.

C’est dans ce contexte que s’inscrit ce projet : concevoir un écosystème d’agents IA spécialisés, capables de transformer un jeu de données brut en un dashboard interactif et un rapport d’analyse, en automatisant l’ensemble du pipeline de l’ingestion des données jusqu’à la publication des résultats.

1.2 Problématique

Face à la nécessité d’automatiser l’analyse de données tout en conservant un contrôle rigoureux sur les actions effectuées par chaque composant du système, plusieurs questions se posent :

- Comment structurer une collaboration efficace entre plusieurs agents IA spécialisés, chacun responsable d’une étape du pipeline d’analyse ?
- Comment garantir que chaque agent n’accède qu’aux outils strictement nécessaires à son rôle, et que chaque action soit traçable ?
- Comment concevoir un système suffisamment générique pour s’adapter à des jeux de données de domaines métier différents (e-commerce, santé, smart city...) sans reconfiguration manuelle ?
- Comment garantir la qualité des données traitées, notamment face à des valeurs manquantes, des doublons ou des anomalies, sans perte d’information exploitable ?

La problématique centrale de ce projet est donc la suivante :

Comment concevoir un système multi-agent orchestré, capable de transformer automatiquement un jeu de données brut en indicateurs de performance, graphiques et rapport exploitable, tout en garantissant la traçabilité et le contrôle des actions de chaque agent via un protocole standardisé ?

1.3 Objectifs du projet

Le projet a pour objectif de développer une plateforme complète d'analyse de données pilotée par des agents IA collaboratifs. Les objectifs spécifiques sont :

- Concevoir un serveur MCP central assurant le contrôle d'accès, l'exécution des outils et la journalisation de chaque action.
- Développer six agents IA spécialisés (Orchestrateur, Data Engineer, Data Scientist, BI/Visualisation, DevOps, Reporter), chacun reposant sur un modèle de langage capable de raisonner et d'appeler des outils de manière autonome.
- Construire un pipeline complet : ingestion et profilage du jeu de données, nettoyage adaptatif, détection automatique du domaine métier, calcul des KPIs, génération de graphiques, publication d'un dashboard et compilation d'un rapport final.
- Valider le système sur un jeu de données réel contenant des problèmes de qualité authentiques (valeurs manquantes, doublons, anomalies).
- Garantir la gratuité d'usage du système en s'appuyant sur des modèles de langage open-source accessibles via une API gratuite (Groq).

Chapitre 2

Fondamentaux : Agents IA et Protocole MCP

2.1 Introduction aux agents IA

Un agent IA, dans le contexte des modèles de langage modernes, désigne un système composé d'un modèle de langage (LLM) auquel est associé un *system prompt* définissant son rôle, ainsi qu'un ensemble d'outils (*tools*) qu'il est autorisé à utiliser pour accomplir une mission. Contrairement à un script informatique classique où la séquence d'instructions est fixée à l'avance par le développeur, un agent IA **décide lui-même**, à chaque étape, quelle action entreprendre en fonction du contexte et du résultat des actions précédentes.

Cette capacité de décision autonome repose sur un mécanisme appelé *tool-use* (ou *function calling*) : le modèle de langage reçoit, en plus du message de l'utilisateur, une description structurée des outils disponibles (nom, description, paramètres attendus). Il peut alors choisir d'appeler un outil plutôt que de répondre directement par du texte, observer le résultat de cet appel, puis décider de la prochaine action.

2.1.1 Principe du *tool-use*

Le fonctionnement d'un agent repose sur une boucle itérative illustrée ci-dessous :

1. L'agent reçoit une mission (ex. : « Nettoie le fichier de données fourni »).
2. Le modèle de langage analyse la mission et décide d'appeler un outil (ex. : `load_dataset`).
3. Le résultat de l'outil est renvoyé au modèle, qui l'intègre à son contexte.
4. Le modèle décide de la prochaine action : appeler un autre outil, ou formuler une réponse finale.
5. Cette boucle se répète jusqu'à ce que le modèle considère la mission accomplie (*stop_reason = end_turn*).

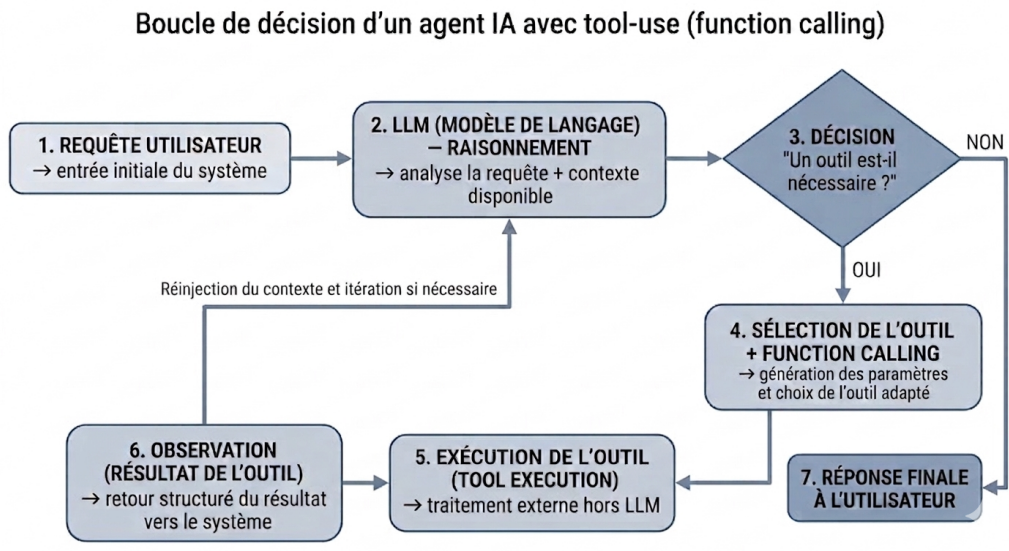


FIGURE 2.1 – Boucle de décision d'un agent IA avec tool-use

2.2 Le protocole MCP (Model Context Protocol)

Le *Model Context Protocol* (MCP) est un protocole standardisé permettant à des agents IA d'accéder à des outils et des ressources externes de manière structurée et contrôlée. Il introduit un serveur central — le **serveur MCP** — qui joue le rôle d'intermédiaire unique entre les agents et les outils qu'ils souhaitent utiliser.

Dans notre système, le serveur MCP assure quatre fonctions essentielles :

- **Registre des outils** : il maintient la liste des outils disponibles ainsi que leur contrat d'entrée/sortie (schéma des paramètres attendus).
- **Contrôle des permissions** : il vérifie, à chaque appel, que l'agent demandeur est autorisé à utiliser l'outil demandé. Un agent ne peut appeler que les outils qui lui sont explicitement attribués.
- **Exécution des outils** : il charge dynamiquement le module Python correspondant à l'outil demandé et exécute la fonction avec les paramètres fournis.
- **Journalisation** : chaque appel d'outil, son résultat, son statut (succès/échec) et son horodatage sont enregistrés dans un fichier de log (`tool_calls.jsonl`), garantissant la traçabilité complète du pipeline.

2.2.1 Pourquoi un serveur MCP plutôt qu'un appel direct ?

Une architecture alternative aurait consisté à laisser chaque agent appeler directement les fonctions Python correspondant aux outils. Cette approche, plus simple à première vue, présente cependant des limites importantes :

- Aucun contrôle centralisé des permissions — un agent pourrait appeler n'importe quel outil, y compris ceux qui ne relèvent pas de son rôle.
- Aucune traçabilité unifiée des actions effectuées par le système.
- Une forte dépendance entre le code d'un agent et l'implémentation exacte des outils, rendant le système difficile à faire évoluer.

Le passage par un serveur MCP centralisé résout ces limites en imposant un point de passage unique, contrôlé et journalisé pour toute interaction entre un agent et un outil.

Architecture du serveur MCP et de ses interactions avec les agents

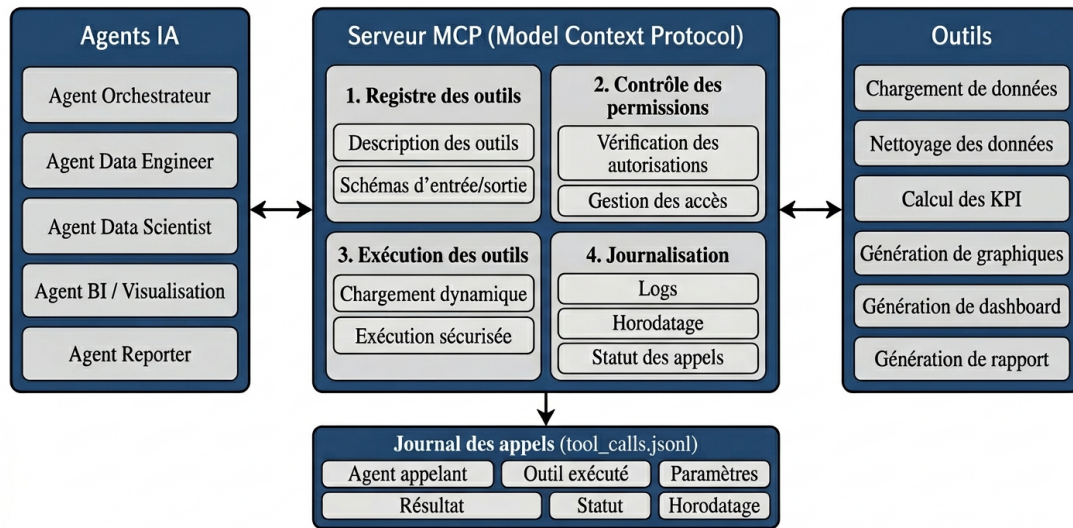


FIGURE 2.2 – Architecture du serveur MCP et de ses interactions avec les agents

2.3 L'orchestration multi-agent

Au-delà du fonctionnement individuel de chaque agent, notre système repose sur une **orchestration hiérarchique** : un composant central, l'Orchestrateur, est responsable de décomposer l'objectif global en tâches élémentaires, de déléguer chaque tâche à l'agent compétent, et de transmettre les résultats d'un agent vers le suivant.

Cette orchestration repose sur trois sous-composants :

- **Le Planner** : détermine, à partir de l'état courant du pipeline, quelle est la prochaine étape à exécuter (par exemple, après le Data Engineer, c'est le Data Scientist qui doit intervenir).
- **Le Router** : associe chaque étape identifiée par le Planner à l'instance d'agent correspondante.
- **L'Engine** : exécute la boucle principale du pipeline, transmet les fichiers produits par un agent au suivant, et gère les erreurs éventuelles.

Ce modèle d'orchestration hiérarchique — où un coordinateur central distribue les tâches plutôt que de laisser les agents négocier entre eux de façon décentralisée (modèle pair-à-pair) — a été retenu car il garantit un déroulement prévisible et reproductible du pipeline, tout en simplifiant le débogage et la traçabilité.

2.4 Choix du modèle de langage : Llama 3.1 via Groq

Le choix du modèle de langage utilisé par les agents repose sur deux contraintes : la nécessité d'un support fiable du *tool-use*, et la gratuité d'utilisation pour un usage

académique intensif. Notre équipe a retenu le modèle **Llama 3.1 8B Instant**, accessible gratuitement via l'API **Groq**, offrant jusqu'à 14 400 requêtes gratuites par jour — largement suffisant pour le développement et les tests du système.

Ce choix s'inscrit dans les recommandations du sujet, qui suggère l'utilisation de modèles *instruction-tuned* supportant le *tool-use*, tels que Llama 3.1 ou Qwen2.5, comme alternative gratuite aux API payantes (OpenAI GPT-4o, Anthropic Claude). L'architecture du système a néanmoins été conçue pour rester indépendante du fournisseur de modèle : la classe **BaseAgent** encapsule la logique de communication avec le LLM, ce qui permettrait de basculer vers un autre fournisseur (Claude, GPT-4o) en ne modifiant qu'un seul fichier.

Chapitre 3

Méthodologie et Conception du Système

3.1 Choix des outils et technologies

La mise en œuvre de ce projet repose sur l'intégration d'un ensemble d'outils technologiques assurant la gestion complète du pipeline d'analyse : de l'ingestion des données à leur traitement par les agents IA, jusqu'à la publication du dashboard. Les outils ont été choisis pour leur compatibilité avec le *tool-use*, leur simplicité d'intégration, et leur gratuité d'usage.

3.1.1 Langages et frameworks

Python Langage principal du projet, utilisé pour l'ensemble du backend : le serveur MCP, les agents, les outils de traitement de données et l'orchestrateur. Sa richesse en bibliothèques de manipulation de données (Pandas) et son support natif des API de *tool-use* en font un choix naturel pour ce type de système.

FastAPI Framework web léger et asynchrone utilisé pour implémenter le serveur MCP. Il expose les routes permettant aux agents d'appeler les outils (POST /call), de consulter la liste des outils disponibles (GET /tools), de vérifier les permissions d'un agent (GET /permissions/{agent}), et de récupérer les journaux d'exécution d'un run (GET /logs/{run_id}).

Échanges entre le serveur MCP, les agents IA et l'interface utilisateur

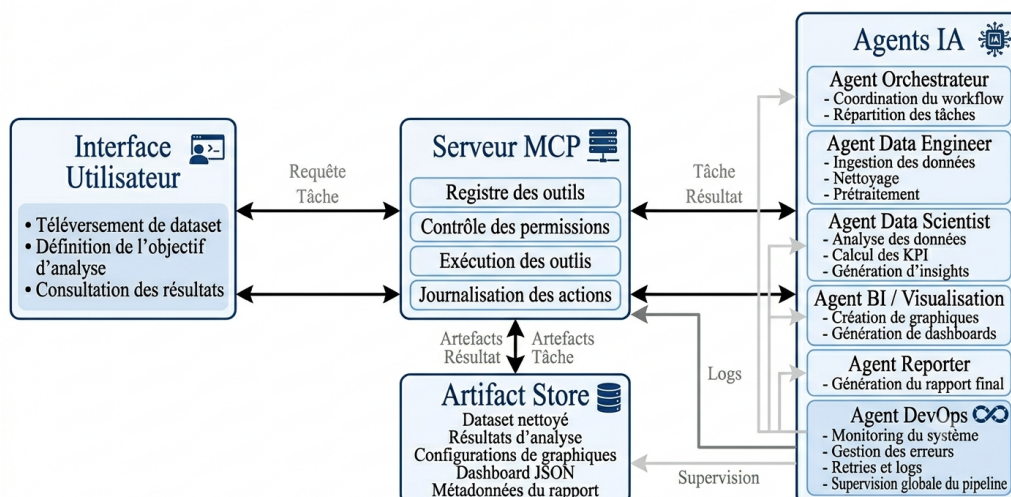


FIGURE 3.1 – Échanges entre le serveur MCP, les agents IA et l'interface utilisateur

3.1.2 Bibliothèques utilisées

Le projet repose sur des bibliothèques Python éprouvées pour le traitement et la visualisation des données :

- **Pandas** — manipulation, nettoyage et transformation des jeux de données tabulaires.
- **Plotly** — génération de graphiques interactifs (barres, lignes, secteurs) exportés au format HTML, intégrés au dashboard final.
- **Pydantic** — définition et validation des modèles de données échangés entre les composants du système (**ToolCall**, **RunState**, **Artifact**).
- **Groq (SDK Python)** — communication avec l'API Groq pour l'inférence du modèle Llama 3.1.
- **python-dotenv** — gestion sécurisée des clés d'API via un fichier `.env` non versionné.

3.1.3 Environnement de développement

Le développement a été réalisé dans un environnement collaboratif structuré :

- **Visual Studio Code**, pour l'édition et l'organisation du code.
- **Git et GitHub**, pour le suivi des versions et la répartition du travail entre les quatre membres de l'équipe.
- **Terminal / uvicorn**, pour le lancement et le test en local du serveur MCP.

3.2 Architecture globale du système

Le système est structuré en quatre couches principales, illustrées par le schéma ci-dessous :

- **Interface utilisateur (UI)** — permet de soumettre un jeu de données, de suivre l'activité des agents en temps réel, et de consulter le dashboard et le rapport final.
- **Couche d'orchestration** — comprend l'Orchestrateur (Engine, Planner, Router) qui pilote le déroulement du pipeline.
- **Serveur MCP** — registre central des outils, gestion des permissions et journalisation.
- **Artifact Store** — système de fichiers organisé par *run*, stockant les données nettoyées, les insights, les graphiques et le rapport final.

Architecture globale d'un système multi-agent IA pour l'analyse de données basé sur MCP

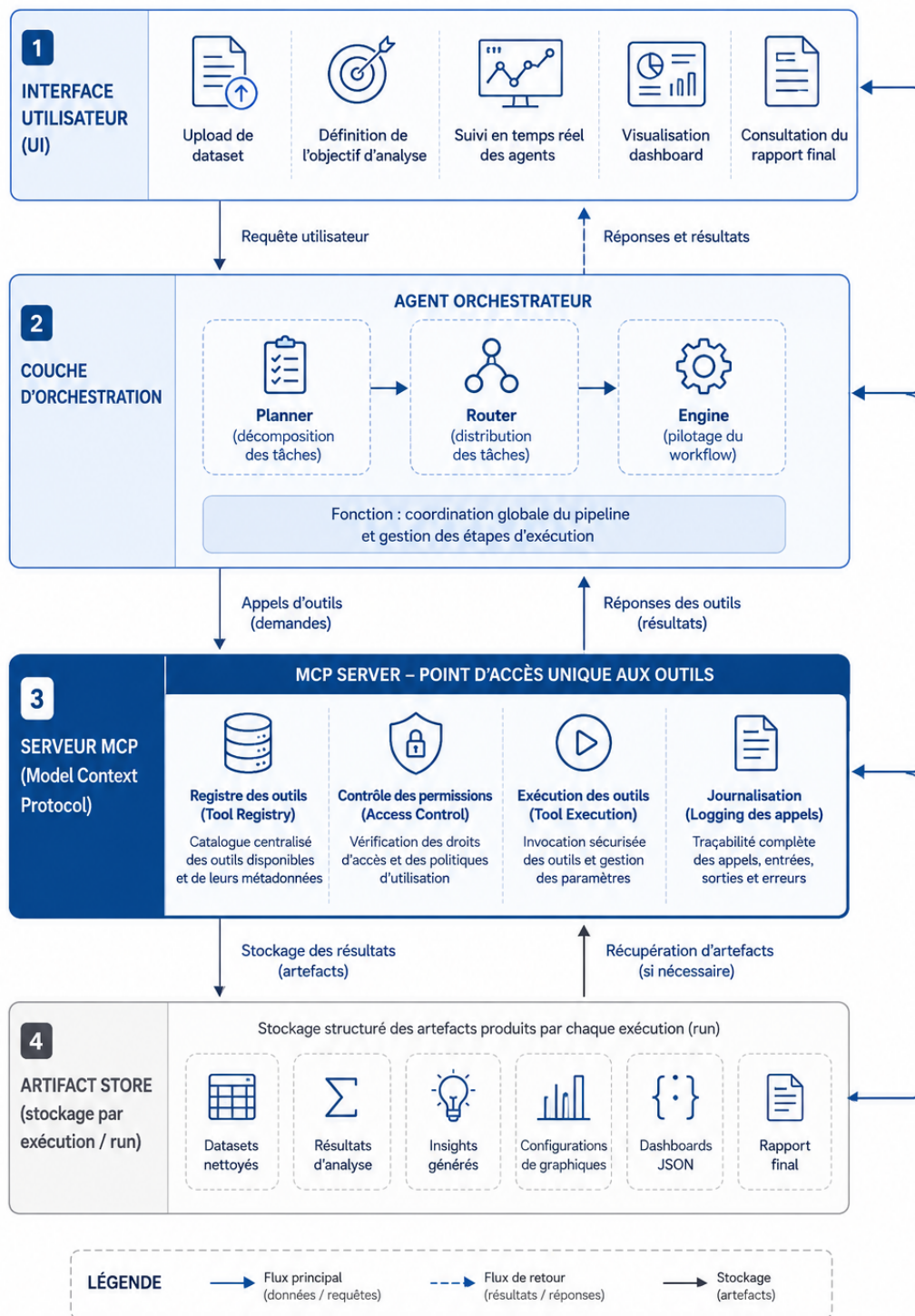


FIGURE 3.2 – Architecture globale du système multi-agent

Les six agents du système

Agent Orchestrateur

L'Orchestrateur est le cerveau central du système. Il reçoit en entrée un fichier de données et un objectif métier, puis organise l'ensemble du pipeline en une séquence coordonnée de tâches : *data_engineer* → *data_scientist* → *bi_agent* → *reporter*.

Responsabilités principales :

- **Décomposition du pipeline** transforme l'objectif métier en tâches atomiques via le *Planner*, en respectant la dépendance entre étapes.
- **Délégation et routage** identifie pour chaque tâche l'agent compétent via le *Router* et instancie l'agent avec le contexte approprié (*run_id*, *dataset_path*, *artifacts*).
- **Supervision et gestion d'erreurs** supervise l'exécution de chaque agent, capture les erreurs et consulte le *DevOpsAgent* pour décider d'une action : *retry* (relancer jusqu'à MAX_RETRIES fois), *skip* (ignorer la tâche et continuer), ou *escalate* (arrêter le pipeline).
- **Gestion d'état partagé** maintient un objet *RunState* qui mémorise les tâches complétées, en cours et en attente, ainsi que tous les artifacts produits.
- **Validation et reporting** valide la sortie finale du pipeline, compile un résumé d'exécution (métadonnées, statut, tâches complétées) et le sauvegarde dans *runs/{run_id}/metad*.

Interactions clés : L'Orchestrateur communique avec :

- Le **Planner** pour déterminer la prochaine tâche à exécuter.
- Le **Router** pour récupérer l'instance de l'agent correspondant.
- Le **DevOpsAgent** pour gérer les erreurs d'exécution et les décisions de *retry/skip/escalate*.
- Chaque **agent métier** (*data_engineer*, *data_scientist*, *bi_agent*, *reporter*) pour exécuter les tâches du pipeline.

Code responsable : La logique de l'Orchestrateur réside dans la classe *Engine* (*app/orchestrator/engine.py*), qui expose une méthode publique *run_pipeline(file_path, objective, run_id)*.

Agent Data Engineer

Le Data Engineer Agent est un agent IA intelligent basé sur une architecture MCP (*Model Context Protocol*), chargé d'automatiser les étapes de préparation et de qualité des données. Il orchestre dynamiquement le chargement, l'analyse et le nettoyage des datasets en combinant des méthodes statistiques (Pandas), des modèles de langage (LLM via Groq) et des règles heuristiques de sécurité.

Responsabilités principales : L'agent exécute un pipeline de trois étapes séquentielles :

- **Étape 1 : Chargement des données** (`load_dataset`) charge des fichiers CSV, JSON, Excel, URLs ou APIs, détecte automatiquement le format, génère un aperçu et mémorise les métadonnées (nombre de lignes, colonnes, types, taille).
- **Étape 2 : Profilage de qualité** (`profile_data`) analyse statistiquement le dataset pour détecter : valeurs manquantes, doublons, valeurs négatives inattendues, colonnes numériques cachées dans du texte, dates détectables, et distributions basiques. Consulte l'API Groq pour une interprétation intelligente et génère un *quality_score* ($0 \rightarrow 1$).
- **Étape 3 : Nettoyage intelligent** (`clean_data`) récupère le profil précédent, demande à Groq des règles de nettoyage structurées en JSON (`drop_duplicates`, `fill_nulls`, `convert_dtype`, `drop_rows_where`, etc.), applique dynamiquement ces règles sur Pandas, et exécute un post-traitement de sécurité (protection des IDs, validation des dates, détection d'outliers, encodage).

Architecture d'intelligence hybride : L'agent combine trois niveaux d'intelligence pour une robustesse maximale :

1. **Niveau statistique** (Pandas) profilage brut, calculs exacts sur les distributions et détection d'anomalies numériques.
2. **Niveau LLM** (Groq *llama-3.1-8b-instant*) détection contextuelle des problèmes de données, génération dynamique de règles de nettoyage sans domaine métier fixe, et interprétation intelligente du contenu des colonnes.
3. **Niveau règles métier** protection heuristique des identifiants, validation des formats de dates, gestion des valeurs aberrantes, et fallback robuste en cas d'absence de réponse LLM.

Processus d'exécution : Le pipeline s'exécute de manière séquentielle. Chaque étape :

- Produit des *artifacts* (fichiers intermédiaires) sauvegardés dans `runs/{run_id}/artifacts/` : `profile.json`, `cleaning_rules.json`, `clean.csv`.
- Communique avec le **MCP Server** pour invoquer les outils spécialisés.
- Retourne un résultat structuré au Moteur (Engine) : statut, chemin du fichier propre, qualité avant/après, et nombre de lignes conservées.

Sortie finale : Le Data Engineer Agent produit :

- **clean.csv** — dataset nettoyé, cohérent et prêt pour l'étape d'analyse (Data Scientist).
- **profile.json** rapport détaillé des problèmes détectés et du *quality_score*.
- **cleaning_rules.json** trace des règles de nettoyage appliquées (audit trail).
- **Métriques** — nombre de lignes avant/après, pourcentage de suppression, qualité brute vs qualité nettoyée.

Interactions clés : L'agent communique avec :

- Le **MCP Server** pour invoquer les trois outils spécialisés (`load_dataset`, `profile_data`, `clean_data`).

- L’**API Groq** (modèle *llama-3.1-8b-instant*) pour générer intelligemment les règles de nettoyage et interpréter le profil.
- L’**Orchestrateur (Engine)** pour signaler succès ou erreur et transférer les artifacts à l’étape suivante (Data Scientist).

Code responsable : La logique du Data Engineer Agent réside dans la classe `DataEngineerAgent` (`app/agents/data_engineer.py`), qui expose une méthode publique `run(file_path)` orchestrant les trois étapes du pipeline via des appels MCP.

Agent Data Scientist

L’Agent Data Scientist est un composant intelligent d’un système multi-agents chargé d’analyser les données nettoyées et transformées afin d’en extraire de la valeur métier exploitable. Il intervient après l’Agent Data Engineer et se charge de transformer les données brutes en insights, KPIs et recommandations structurées.

Rôle principal : L’Agent Data Scientist exécute les tâches suivantes :

- Analyser les datasets nettoyés et valider leur cohérence.
- Calculer des KPIs (indicateurs clés de performance) métier-agnostiques et domaine-spécifiques.
- Détecter automatiquement le domaine métier du dataset (e.g., *passenger_survival*, *sales_forecast*, *customer_churn*).
- Évaluer la qualité des données via des scores de complétude et de cohérence.
- Générer des insights contextuels et recommandations.
- Proposer des visualisations adaptées (*chart_hints*).

Fonctionnement : Deux cas de gestion des colonnes L’agent adapte son comportement selon la nature des colonnes du dataset :

Cas 1 : Dataset avec colonnes visibles (non-anonyme) Lorsque les colonnes du fichier CSV possèdent des noms explicites et compréhensibles (par exemple **Revenue**, **Country**, **Price**, **Date**), l’agent détecte qu’il s’agit d’un dataset non-anonyme. Dans ce cas :

1. L’agent **ne déclenche pas** d’étape de renommage de colonnes.
2. Il passe **directement** à l’appel de la fonction `run_analysis`, qui orchestre l’analyse complète : détection du domaine métier, génération des KPIs via le modèle LLM (Groq), exécution des calculs statistiques avec Pandas, production des insights, alertes et score de qualité.
3. L’agent enrichit simplement les résultats en ajoutant des suggestions de graphiques (*chart_hints*) basées sur les types de KPIs détectés.
4. Enfin, il retourne le rapport complet (`insights.json`) à l’Orchestrateur.

Cas 2 : Dataset avec colonnes anonymes Lorsque les colonnes sont peu explicites ou génériques (par exemple `col1`, `col2`, `feature_3`, 1, 2), l'agent détecte qu'il s'agit d'un dataset anonyme en analysant la structure des noms de colonnes. Dans ce cas :

1. L'agent déclenche une **étape de résolution des noms de colonnes** avant l'analyse proprement dite.
2. Il utilise d'abord le modèle LLM (Groq) pour proposer des alias intelligents (par exemple `col1` → `revenue`, `col2` → `price`) en se basant sur les statistiques descriptives des colonnes (moyennes, distributions, types de données).
3. Si cette étape échoue (par exemple, si le LLM ne peut pas inférer le sens), l'agent applique une **méthode heuristique** basée sur les types de données détectés par Pandas et les distributions observées, pour inférer des alias approximatifs.
4. Une fois les colonnes renommées ou interprétées, le processus **continue normalement** avec l'appel à `run_analysis`, qui génère les KPIs, les insights et les alertes.
5. Enfin, les résultats sont enrichis avec les `chart_hints` et sauvegardés dans `insights.json`.

Architecture du pipeline d'analyse : Le pipeline suit une séquence logique :

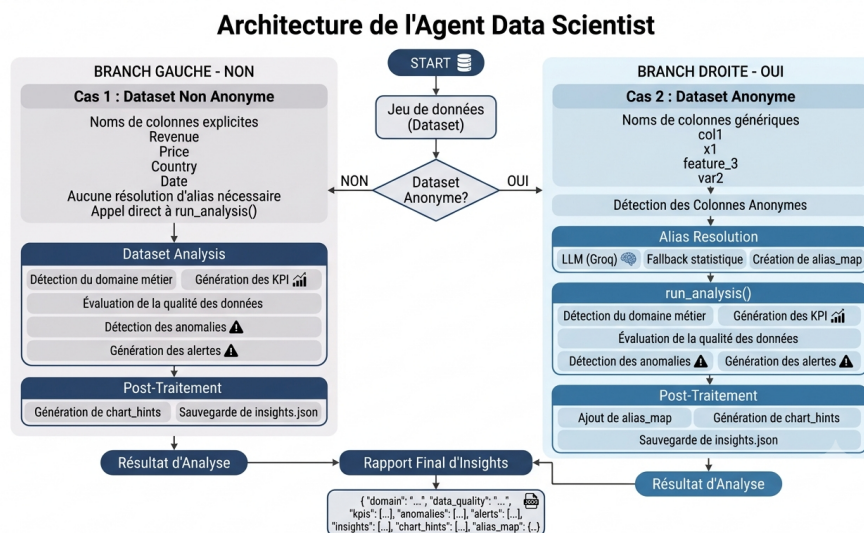


FIGURE 3.3 – Pipeline du Data Scientist Agent : gestion des colonnes anonymes vs non-anonymes

Sorties produites : L'Agent Data Scientist génère un rapport structuré (`insights.json`) contenant :

- **data_quality** — score de qualité (0 → 1) et taux de complétude des données.
- **kpis** — dictionnaire de métriques métier : chiffre d'affaires (CA), taux de conversion, moyennes, totaux, min/max, etc.
- **anomalies** — liste des valeurs atypiques détectées (outliers, valeurs incohérentes).
- **alertes** — warnings critiques et recommandations d'action.
- **insights** — interprétations textuelles générées par le LLM sur les tendances, corrélations et découvertes majeures.
- **chart_hints** — suggestions de graphiques (bar, line, pie, scatter, heatmap) basées sur les types de KPIs et la structure des données.

- **domain** — domaine métier détecté automatiquement (e.g., *passenger_survival*, *sales_forecast*).
- **alias_map** — (si colonnes anonymes) mapping des noms de colonnes originaux vers les alias résolus.

Processus d'exécution : L'agent exécute la séquence suivante :

1. Charge le fichier `clean.csv` produit par le Data Engineer.
2. Analyse les noms de colonnes pour détecter l'anonymat.
3. Si anonyme : déclenche la résolution via LLM ou heuristique.
4. Appelle `run_analysis(dataset, run_id)` qui :
 - Détecte le domaine métier.
 - Génère des KPIs adaptés au domaine.
 - Calcule les statistiques via Pandas.
 - Interroge Groq pour les insights textuels.
5. Enrichit les résultats avec des *chart_hints*.
6. Sauvegarde dans `runs/{run_id}/artifacts/insights.json`.

Interactions clés : L'agent communique avec :

- Le **MCP Server** pour invoquer l'outil `run_analysis`.
- L'**API Groq** (modèle *llama-3.1-8b-instant*) pour générer intelligemment les KPIs, déterminer le domaine métier, créer les alias de colonnes (cas anonyme), et produire les insights textuels.
- L'**Orchestrateur (Engine)** pour signaler succès/erreur et transférer les artifacts à l'étape suivante (BI Agent).

Code responsable : La logique du Data Scientist Agent réside dans la classe `DataScientistAgent` (`app/agents/data_scientist.py`), qui expose une méthode publique `run(step, context)` orchestrant la détection d'anonymat, la résolution des colonnes, et l'invocation de `run_analysis` via le MCP Server.

Agent BI (Business Intelligence)

Le BI Agent est un agent intelligent adaptatif chargé de transformer les résultats analytiques produits par le Data Scientist (généralement contenus dans un fichier `insights.json`) en un dashboard décisionnel complet, structuré et interactif. Il agit comme une couche de visualisation et d'interprétation business dans l'architecture multi-agents, en servant de traducteur entre données analytiques et visualisations exploitables pour la prise de décision.

Rôle principal : L'Agent BI exécute les tâches suivantes :

- Lire et interpréter automatiquement le fichier `insights.json` produit par le Data Scientist.
- Extraire les KPI importants et valider leur cohérence.

- Détecter les séries de données exploitables pour la visualisation.
- Générer des graphiques adaptés automatiquement selon le type de données et le domaine métier détecté.
- Construire un dashboard dynamique multi-sections regroupant KPI cards et visualisations.
- Publier un dashboard final en format HTML interactif, prêt pour la consultation et la prise de décision.

Pipeline d'exécution : 4 étapes principales Le BI Agent suit une séquence logique composée de quatre étapes majeures :

Étape 1 : Lecture des insights L'agent localise et charge le fichier `insights.json` produit par le Data Scientist. Il valide et extrait les données suivantes :

- **kpis** dictionnaire des indicateurs clés de performance (CA, taux, moyennes, etc.).
- **data_quality** — score global et taux de complétude.
- **insights** interprétations textuelles et recommandations.
- **chart_hints** suggestions de types de graphiques fournies par le Data Scientist (bar, line, pie, scatter).
- **domain** — domaine métier détecté (e.g., *business*, *finance*, *healthcare*).

Étape 2 : Construction du modèle dashboard L'agent transforme les données brutes en un modèle structuré contenant :

- **KPI cards** représentations visuelles des métriques principales (total, moyenne, score de qualité, etc.).
- **Séries exploitables** — données numériques groupées par catégorie pour les graphiques.
- **Domaine détecté** utilisé pour adapter le design et les recommandations du dashboard.
- **Métrique principale** — sélection automatique du KPI le plus significatif selon sa pertinence métier.

Étape 3 : Génération des visualisations L'agent génère automatiquement des graphiques adaptés en :

1. **DéTECTANT les types de graphiques** — bar chart, line chart, pie chart, scatter plot, en fonction de la nature des données (catégorique, temporelle, numérique).
2. **Priorisant les KPI importants** — ranking automatique selon leur pertinence métier et leur impact sur les décisions.
3. **Intégrant les chart_hints** — respecte les suggestions de visualisation fournies par le Data Scientist.
4. **Créant jusqu'à 12 graphiques maximum** — limite configurable pour éviter une surcharge visuelle.
5. **Appliquant un nettoyage des données** — normalisation, gestion des valeurs manquantes, arrondis intelligents.

Étape 4 : Publication du dashboard L'agent finalise et expose le dashboard via :

- **Appel à `generate_chart`** — invoque l'outil MCP pour créer chaque graphique (format Plotly HTML).
- **Appel à `publish_dashboard`** — assemble tous les graphiques en un dashboard HTML complet et interactif.
- **Création d'un `handoff` file** — génère `bi_agent_handoff.json` contenant la liste des graphiques générés, les seuils d'alerte et les recommandations pour l'étape suivante (Reporter).
- **Retour des artefacts** — chemin du dashboard final (`dashboard.html`), nombre de graphiques générés, et chemins de `handoff`.

Architecture d'intelligence intégrée : Le BI Agent intègre plusieurs capacités d'intelligence pour une adaptation complète à différents types de datasets :

- **Détection automatique de séries numériques** — identifie et extrait les colonnes exploitables pour les graphiques parmi les KPIs.
- **Choix automatique du type de graphique** — mappe chaque type de données vers le graphique le plus approprié (bar pour catégories, line pour tendances temporelles, pie pour proportions).
- **Ranking intelligent des KPI** — classe les KPI selon leur importance métier estimée, en privilégiant les indicateurs de synthèse (total, moyenne) et les ratios critiques.
- **Nettoyage et normalisation** — applique des transformations (arrondi, format monétaire, pourcentage) selon le contexte métier.
- **Fallback robuste** — si les `chart_hints` sont absents ou les données incomplètes, l'agent applique une heuristique par défaut basée sur les types pandas et les distributions observées.
- **Support des hints du Data Scientist** — intègre et respecte les suggestions de visualisation fournies via `chart_hints`.

Sorties produites : Le BI Agent génère les artefacts suivants :

- **`dashboard.html`** — dashboard interactif final contenant KPI cards, graphiques et titre du projet, prêt à être consulté dans un navigateur.
- **Graphiques individuels** — jusqu'à 12 fichiers Plotly HTML générés et incorporés au dashboard.
- **`bi_agent_handoff.json`** — fichier de synthèse contenant :
 - liste complète des graphiques générés.
 - seuils d'alerte détectés (e.g., KPI hors limites).
 - recommandations pour la phase reporting.
- **Rapport structuré** — dictionnaire retourné à l'Orchestracteur contenant le statut de succès, les chemins des artefacts, et le nombre de graphiques générés.

Processus d'exécution : L'agent exécute la séquence suivante :

1. Charge le fichier `insights.json` produit par le Data Scientist.
2. Valide et extrait les KPIs, `chart_hints` et domaine métier.
3. Construit un modèle dashboard interne contenant KPI cards et séries pour graphiques.
4. Pour chaque graphique à générer :
 - Détecte le type de graphique approprié (bar, line, pie, scatter).
 - Nettoie et normalise les données.
 - Invoque `generate_chart` via le MCP Server.
5. Assemble tous les graphiques en un seul dashboard HTML via `publish_dashboard`.
6. Crée le fichier de handoff `bi_agent_handoff.json`.
7. Sauvegarde dans `runs/{run_id}/artifacts/`.

Interactions clés : L'agent communique avec :

- Le **MCP Server** pour invoquer les deux outils spécialisés : `generate_chart` (création des graphiques Plotly) et `publish_dashboard` (assemblage HTML final).
- Le **Data Scientist Agent** — consomme les insights, KPIs et `chart_hints` produits.
- L'**Orchestrateur (Engine)** — retourne le chemin du dashboard final et signale le succès ou l'erreur.
- Le **Reporter Agent** — fournit le dashboard et les KPI clés pour l'étape de génération du rapport final.

Position dans le système multi-agents : Le BI Agent s'insère entre le Data Scientist et le Reporter dans la chaîne de traitement :

Data Engineer → Data Scientist → **BI Agent** → Reporter

Il transforme les insights analytiques bruts en visualisations métier prêtes à la consultation et facilite ainsi la prise de décision.

Code responsable : La logique du BI Agent réside dans la classe `BIAgent` (`app/agents/bi_agent.py`) qui expose une méthode publique `run(step, context)` orchestrant la lecture des insights, la construction du modèle dashboard, la génération des graphiques, et la publication du dashboard final via le MCP Server.

Agent DevOps

L'Agent DevOps est une couche transverse de fiabilité et monitoring chargée de surveiller, exécuter et sécuriser les appels de fonctions (tools) des autres agents. Il garantit la robustesse, la traçabilité et la récupération en cas d'erreur.

Rôle principal :

- Exécuter les outils des agents de manière sécurisée avec timeout et isolation.
- Gérer automatiquement les retries (jusqu'à 3 tentatives) avec backoff exponentiel.
- Appliquer des fallbacks automatiques en cas d'échec total.
- Journaliser tous les événements d'exécution (succès, erreurs, retries).
- Analyser les incidents via Claude pour recommander les prochaines actions.

Pipeline d'exécution :

1. Exécute une fonction avec timeout (60s), isolation et retry automatique.
2. Gère les erreurs : retry simple, retry réussi, fallback ou escalade.
3. Enregistre chaque exécution dans `runs/{run_id}/devops/` avec statut, tentatives, durée, erreur.
4. Envoie les incidents à Claude (modèle *claude-sonnet-4*) pour analyse et recommandation.
5. Valide les outputs et génère un rapport de session agrégé.

Sorties produites :

- **Logs JSON** — fichiers d'exécution dans `devops/`.
- **devops_summary.json** — rapport agrégé avec statistiques et recommandations LLM.
- **audit_trail.json** — historique complet pour audit.

Interactions clés :

- Supervise tous les appels de tools via le MCP Server.
- Communique avec l'API Claude pour l'analyse intelligente.
- Envoie les signaux d'escalade à l'Orchestrateur.

Code responsable : La classe `DevOpsAgent` (`app/agents/devops_agent.py`) expose : `execute_tool()`, `handle_failure()`, `generate_session_report()`.

Agent Reporter

L'Agent Reporter est l'agent final d'un pipeline multi-agents d'analyse de données. Il est responsable de la génération du rapport final structuré (PDF) en agrégeant et en synthétisant tous les résultats produits par les autres agents. Il joue le rôle de couche de communication finale entre le système analytique et l'utilisateur métier.

Rôle principal :

- Collecter tous les artefacts du run (KPIs, dashboard, data quality, alertes).
- Synthétiser les résultats analytiques en un rapport compréhensible et structuré.
- Générer un rapport PDF final contenant résumé exécutif, insights clés et recommandations.
- Sauvegarder et enregistrer le rapport dans les artefacts du run.
- Fournir un résumé exécutif court prêt pour la présentation.

Pipeline d'exécution :

1. Récupère les informations du run : `run_id`, objectif, et artefacts des agents précédents (KPIs, dashboard, data quality, alertes).
2. Construit un prompt structuré et interroge le LLM (Groq) pour générer le contenu du rapport.
3. Invoque l'outil `compile_report` via le MCP Server pour assembler le PDF final.
4. Enregistre le rapport avec `log_artifact` dans les artefacts du run.
5. Parse le JSON de réponse et applique un fallback si nécessaire (génération d'un chemin par défaut, troncature du résumé).

Contenu du rapport PDF généré :

- **Résumé exécutif** — synthèse du run et résultats clés.
- **Data Quality Score** — qualité des données analysées et problèmes détectés.
- **KPIs business principaux** — indicateurs clés extraits par le Data Scientist.
- **Alertes** — warnings et seuils critiques détectés.
- **Référence au dashboard** — lien ou intégration du dashboard interactif généré par le BI Agent.
- **Recommandations** — actions suggérées basées sur les insights.

Sorties produites :

- **report.pdf** — rapport final structuré contenant tous les résultats et recommandations.
- **Rapport JSON** — dictionnaire retourné à l'Orchestrator avec chemin du PDF et résumé exécutif.

Interactions clés :

- Le **MCP Server** pour invoquer `compile_report` et `log_artifact`.
- L'**API Groq** (modèle *llama-3.1-8b-instant*) pour générer intelligemment le contenu du rapport.
- Tous les artefacts produits par les agents précédents (Data Engineer, Data Scientist, BI Agent, DevOps Agent).
- L'**Orchestrator (Engine)** pour signaler la fin du pipeline.

Position dans le pipeline multi-agents : Le Reporter Agent est le point de sortie final du système :

Data Engineer → Data Scientist → BI Agent → **Reporter Agent**

Il transforme les résultats analytiques en un rapport métier prêt pour la prise de décision et la communication.

Code responsable : La classe `ReporterAgent` (`app/agents/reporter.py`) expose la méthode `run(step, context)` orchestrant la collecte des artefacts, la génération du rapport via le LLM, et l'invocation des outils du MCP Server.

Conclusion

Agent	Responsabilité
Orchestrateur (CEO)	Reçoit l'objectif, décompose le pipeline en tâches, délègue à chaque agent, surveille l'avancement et valide la sortie finale.
Data Engineer	Ingère le jeu de données, le profile (schéma, valeurs manquantes, distributions), le nettoie et le transforme.
Data Scientist	Exécute l'analyse statistique, détecte le domaine métier, calcule les KPIs et produit un résumé structuré des insights.
BI / Visualisation	Sélectionne les types de graphiques pertinents à partir des insights, génère les visualisations et publie le dashboard.
DevOps	Gère l'environnement d'exécution, traite les erreurs d'outils, relance les étapes échouées et garantit l'aboutissement du pipeline.
Reporter	Agrège les sorties de tous les agents dans un rapport final synthétique (méthodologie, résultats clés, référence au dashboard).

TABLE 3.1 – Rôles et responsabilités des agents du système

3.2.1 Les neuf outils MCP

Dans l'architecture proposée, les agents ne disposent pas d'un accès direct aux jeux de données ni aux ressources du système. Toutes les interactions sont réalisées à travers un ensemble d'outils standardisés enregistrés sur le serveur MCP (Model Context Protocol). Cette approche permet de centraliser les opérations, d'assurer la traçabilité des actions effectuées par les agents et de garantir une meilleure séparation entre la logique métier des agents et les fonctions techniques du système.

Chaque outil MCP encapsule une fonctionnalité spécifique et expose une interface uniforme permettant aux agents d'exécuter des tâches précises telles que le chargement des données, l'analyse statistique, la génération de visualisations ou encore la production de rapports. Lorsqu'un agent souhaite réaliser une opération, il formule une requête au serveur MCP qui identifie l'outil approprié, exécute la fonction demandée et retourne le résultat à l'agent.

Dans le cadre de ce projet, neuf outils MCP ont été développés afin de couvrir l'ensemble des besoins du pipeline d'analyse de données, depuis l'acquisition du jeu de données jusqu'à la génération du rapport final. Le tableau 3.2 récapitule ces différents outils ainsi que leur rôle principal au sein du système.

Outil	Fonction
load_dataset	Charge un fichier CSV/Excel/JSON et retourne son schéma ainsi qu'un aperçu des données.
profile_data	Calcule un profil de qualité des données : valeurs manquantes, doublons, distributions, anomalies détectées.
quality_check	Calcule un score de qualité global du jeu de données.
clean_data	Applique un ensemble de règles de nettoyage (génériques ou proposées par le LLM) : suppression de doublons, imputation des valeurs manquantes, conversion de types, création de colonnes dérivées.
run_analysis	Détecte le domaine métier du jeu de données et calcule les KPIs pertinents (somme, moyenne, regroupements, corrélations...).
generate_chart	Génère un graphique interactif (barres, lignes, secteurs) au format HTML à partir d'une spécification de données.
publish_dashboard	Assemble plusieurs graphiques en un dashboard HTML unique.
log_artifact	Enregistre une entrée structurée dans le journal d'exécution du run courant.
compile_report	Compile un rapport final (HTML/PDF) à partir des résultats produits par l'ensemble du pipeline.

TABLE 3.2 – Liste des outils enregistrés sur le serveur MCP

3.2.2 Table des permissions

Le contrôle d'accès du serveur MCP repose sur une table de permissions statique, définissant pour chaque agent l'ensemble exact des outils qu'il est autorisé à appeler. Toute tentative d'appel à un outil non autorisé est rejetée avec une erreur HTTP 403.

Agent	Outils autorisés
orchestrator	* (accès à tous les outils)
data_engineer	load_dataset, profile_data, clean_data, log_artifact
data_scientist	run_analysis, log_artifact
bi_agent	generate_chart, publish_dashboard, log_artifact
reporter	compile_report, log_artifact
devops	log_artifact

TABLE 3.3 – Table des permissions par agent

3.3 Implémentation des agents

3.3.1 La classe BaseAgent

Tous les agents héritent d'une classe commune **BaseAgent**, qui encapsule la logique partagée : la connexion à l'API Groq, la boucle de décision (*tool-use loop*), et la communication avec le serveur MCP. Cette mutualisation permet à chaque agent spécialisé de ne définir que son *system prompt* et la séquence d'outils qu'il privilégie, sans dupliquer la logique d'orchestration interne.

```
class BaseAgent:
    agent_name = "base_agent"
    system_prompt = "Tu es un agent IA."
    HEAVY_TOOLS = {"clean_data", "profile_data", "run_analysis"}
    def __init__(self, run_id: str = ""):
        self.run_id = run_id
        self.client = groq.Groq(api_key=os.getenv("GROQ_API_KEY"))
        self.model = "llama-3.1-8b-instant"
        print(f" Groq - modèle : {self.model} - agent : {self.agent_name}")
    def _call_llm(self, messages: list, tools: list):
        all_messages = [
            {"role": "system", "content": self.system_prompt}
        ] + self._convert_messages(messages)
        groq_tools = [
            {
                "type": "function",
                "function": {
                    "name": t["name"],
                    "description": t["description"],
                    "parameters": t["input_schema"]
                }
            }
            for t in tools
        ]
        return GroqResponse(
            self.client.chat.completions.create(
                model=self.model,
```

FIGURE 3.4 – Boucle de décision dans BaseAgent (extrait)

3.3.2 Le serveur MCP

Le serveur MCP, implémenté avec FastAPI, expose une route centrale POST `/call` par laquelle transitent tous les appels d'outils. Cette route applique systématiquement quatre étapes : vérification de la permission, résolution dynamique du module Python correspondant à l'outil, exécution, puis journalisation du résultat.

```
@app.post("/call")
def call_tool(req: ToolRequest):
    if not is_authorized(req.agent, req.tool):
        raise HTTPException(403, f"'{req.agent}' ne peut pas appeler '{req.tool}'")
    module_path = get_tool_module(req.tool)
    if not module_path:
        raise HTTPException(404, f"Outil '{req.tool}' introuvable")
    try:
        module = importlib.import_module(module_path)
        func = getattr(module, req.tool)
        raw = func(**req.params)
        result = raw[1] if req.tool in TUPLE_TOOLS else raw
        success = True
        error = ""
        print(f"OUTIL OK : {req.tool}")
    except Exception as e:
        result = {}
        success = False
        error = str(e)
        print(f"ERREUR OUTIL : {e}")
```

FIGURE 3.5 – Route principale du serveur MCP (extrait)

```

        print(f"ERREUR OUTIL : {e}")
    if req.run_id:
        store.log_tool_call(req.run_id, ToolCall(
            agent_name = req.agent,
            tool_name = req.tool,
            input      = req.params,
            output     = result if isinstance(result, dict) else {},
            success    = success,
            error      = error,
            timestamp  = ""
        ))
    if not success:
        raise HTTPException(500, error)
    return {"result": result}

```

FIGURE 3.6 – Route principale du serveur MCP (suite)

3.3.3 Orchestration du pipeline (Engine)

L'Engine constitue le cœur de l'orchestration : il exécute une boucle principale qui, à chaque itération, interroge le Planner pour connaître la prochaine étape, obtient l'agent correspondant via le Router, lui transmet le fichier produit par l'étape précédente, puis enregistre le résultat avant de passer à l'étape suivante.

```

class Engine:
    """
    La boucle principale du pipeline.
    Ordre : data_engineer → data_scientist → bi_agent (skip)
    """
    def __init__(self):
        self.planner = Planner()
        self.router = Router()
        self.devops_agent = None

    def run(self, file_path: str, run_id: str) -> dict:
        print(f"\n{'*' * 55}")
        print(f"ENGINE DEMARRE - run_id : {run_id}")
        print(f"  Fichier : {file_path}")
        print(f"{'*' * 55}")
        state = RunState(run_id=run_id, file_path=file_path)
        print(f"[Engine] Tâches à faire : {state.pending}\n")
        context = {
            "run_id": run_id,
            "dataset_path": file_path,
            "artifacts": {},
        }
        while not state.is_finished():
            task = self.planner.next_task(state)
            if not task:
                break
            state.current = task
            print(f"\n[Engine] Lancement : {task}")

            gine:
            23 run(self, file_path: str, run_id: str) -> dict:
            40 state.current = task
            41 print(f"\n[Engine] Lancement : {task}")
            42 agent = self.router.get_agent(task, run_id)
            43 if agent is None:
            44     print(f"[Engine] Skip '{task}' - agent non disponible\n")
            45     state.pending.remove(task)
            46     state.completed.append(task + "_skipped")
            47     continue
            48 try:
            49     if task == "data_engineer":
            50         result = agent.run(file_path)
            51         if result.get("status") == "success":
            52             clean_path = result["clean_path"]
            53             state.mark_done(task, {
            54                 "clean_csv": clean_path,
            55                 "quality_score": str(result.get("quality score", "")),
            56                 "final_rows": str(result.get("final_rows", "")),
            57             })
            58             context["artifacts"]["last_file"] = clean_path
            59             context["artifacts"]["data_engineer"] = result
            60             print(f"[Engine] '{task}' termine ✓")
            61             print(f"[Engine] Artifact : {clean_path}")
            62         else:
            63             error = result.get("error", "Erreur inconnue")
            64             print(f"[Engine] ERREUR dans '{task}' : {error}")
            65             state.errors.append(f"{task}: {error}")
            66             break

```

FIGURE 3.7 – Boucle principale de l'Engine (extrait)

3.4 Jeu de données et préparation des données

3.4.1 Présentation du dataset

Pour valider le système sur un cas d'usage réel, l'équipe a retenu le jeu de données public **Online Retail II**, mis à disposition par le UCI Machine Learning Repository. Ce dataset regroupe les transactions réelles d'un détaillant britannique en ligne sur la période de décembre 2009 à décembre 2011.

Caractéristique	Valeur
Source	UCI Machine Learning Repository
Format initial	Excel (.xlsx), deux feuilles (2009–2010 et 2010–2011)
Nombre de lignes	1 067 371 transactions
Colonnes	Invoice, StockCode, Description, Quantity, InvoiceDate, Price, Customer ID, Country
Période couverte	Décembre 2009 – Décembre 2011

TABLE 3.4 – Caractéristiques du jeu de données Online Retail II

Ce jeu de données a été choisi pour sa richesse en problèmes de qualité authentiques, permettant de démontrer concrètement le travail du Data Engineer : valeurs manquantes, annulations de commandes, quantités négatives et prix invalides.

3.4.2 Problèmes de qualité identifiés

Le profilage initial du jeu de données, réalisé par l’outil `profile_data`, a permis d’identifier les anomalies suivantes :

Problème	Lignes concernées	Pourcentage
Factures annulées (préfixe « C »)	19 494	1,83%
Quantités négatives	22 950	2,15%
Prix nuls ou négatifs	6 207	0,58%
Customer ID manquant	243 007	22,77%

TABLE 3.5 – Problèmes de qualité détectés dans le dataset brut

3.4.3 Stratégie de nettoyage adaptative

Plutôt que d’appliquer des règles de nettoyage fixes et codées en dur, l’outil `clean_data` interroge le modèle de langage en lui fournissant un profil compact du jeu de données (noms de colonnes, types, taux de valeurs manquantes). Le modèle propose alors une liste de règles de nettoyage génériques (suppression de lignes selon une condition, imputation de valeurs manquantes, conversion de types, création de colonnes dérivées), qui sont ensuite validées et appliquées par le code Python. En cas d’indisponibilité du modèle, un *fallback* heuristique applique des règles par défaut fondées sur le nom et le type de chaque colonne.

Un traitement particulier est appliqué aux colonnes identifiantes (telles que **Customer ID**) : plutôt que de supprimer les lignes pour lesquelles cette valeur est manquante — ce qui aurait entraîné la perte de 22,77% du jeu de données — le système remplace ces valeurs par la mention "Anonymous", préservant ainsi l’intégralité des transactions pour le calcul des indicateurs business.

3.4.4 Détection adaptative du domaine métier

Une caractéristique notable du système est sa capacité à s’adapter à des jeux de données de domaines métier variés, sans configuration préalable. L’outil `run_analysis` transmet au modèle de langage un profil compact des colonnes du jeu de données nettoyé

(noms, types, statistiques de base), et lui demande de détecter **librement** le domaine métier (sans liste fermée de catégories) ainsi qu’un plan de KPIs pertinents pour ce domaine. Ce plan est ensuite validé (vérification de l’existence des colonnes référencées, protection contre les formules invalides) puis exécuté par Pandas.

Dans le cas du jeu de données Online Retail II, le système a correctement identifié le domaine *e-commerce* et généré un plan de KPIs adapté (chiffre d’affaires total, répartition par pays, produits les plus vendus, panier moyen). En cas d’indisponibilité du modèle de langage, un *fallback* universel calcule des statistiques descriptives génériques (moyenne, médiane, écart-type, corrélations, détection d’anomalies par la méthode de l’écart inter-quartile) sur l’ensemble des colonnes numériques, garantissant que le pipeline aboutit même sans accès au LLM.

Chapitre 4

Pipeline et Résultats Obtenus

4.1 Déroutement du pipeline complet

Le pipeline complet du système s'exécute selon la séquence suivante, chaque étape produisant un artefact consommé par l'étape suivante :

1. **Data Engineer** : ingestion du fichier brut (`online_retail_II.csv`) via `load_dataset`, profilage qualité via `profile_data`, puis nettoyage via `clean_data`, produisant `clean.csv`.
2. **Data Scientist** : analyse du fichier nettoyé via `run_analysis`, détection du domaine métier et calcul des KPIs, produisant `insights.json`.
3. **BI Agent** : génération des graphiques via `generate_chart` à partir des `chart_hints` fournis par le Data Scientist, puis assemblage du dashboard via `publish_dashboard`.
4. **Reporter** : compilation du rapport final via `compile_report`, agrégeant la méthodologie, les résultats et la référence au dashboard.

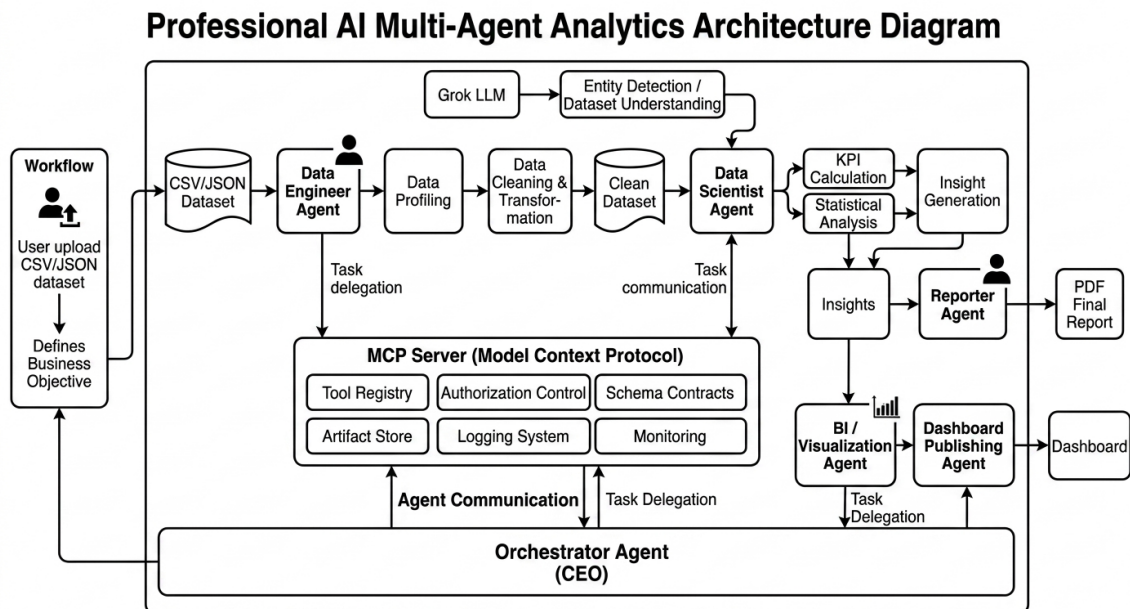


FIGURE 4.1 – Pipeline complet du système, de l'ingestion à la publication du rapport

4.2 Résultats du Data Engineer

L'exécution du Data Engineer sur le jeu de données complet a permis d'obtenir les résultats suivants :

Indicateur	Valeur
Lignes chargées (brutes)	1 067 371
Lignes après nettoyage	1 041 670
Taux de conservation	97,6%
Score de qualité des données	0,954 / 1,0
Colonnes créées	Sales, Year, Month, YearMonth

TABLE 4.1 – Résultats du nettoyage par le Data Engineer

4.3 Résultats du Data Scientist

Le Data Scientist a détecté automatiquement le domaine *e-commerce* et calculé les indicateurs de performance suivants à partir du fichier nettoyé :

KPI Business	Valeur calculée
Chiffre d'affaires total	20 972 594,55 £
Premier pays par CA	Royaume-Uni (17 870 977,76 £)
Produit le plus vendu	REGENCY CAKESTAND 3 TIER (344 563,25 £)
Nombre de clients uniques	5 879
Panier moyen par commande	523,31 £
Nombre de commandes	40 077
Taux d'annulation	0,0% (après nettoyage)

TABLE 4.2 – KPIs calculés sur le jeu de données Online Retail II

4.3.1 Top 5 pays par chiffre d'affaires

Pays	CA (£)
United Kingdom	17 870 977,76
EIRE	664 431,78
Netherlands	554 232,34
Germany	431 262,46
France	356 944,60

TABLE 4.3 – Répartition du chiffre d'affaires par pays (top 5)

4.3.2 Chart hints transmis au BI Agent

Pour permettre au BI Agent de générer les bons graphiques sans avoir à ré-analyser les données, le Data Scientist transmet une liste de *chart_hints* suggérant le type de graphique et la donnée associée :

Identifiant	Type	Titre
chart1	bar	CA total par mois
chart2	pie	CA total par pays

TABLE 4.4 – Chart hints générés par le Data Scientist

4.4 Tests de validation du serveur MCP

Afin de garantir le bon fonctionnement de l'ensemble du pipeline, une série de tests a été réalisée directement sur le serveur MCP via des requêtes HTTP (`curl`). Le tableau 4.5 résume les résultats obtenus.

Test	Commande	Résultat obtenu	Statut
Connexion Groq	Appel API direct	Réponse du modèle reçue	OK
Santé du serveur	GET /health	{"status": "ok"}	OK
Liste des outils	GET /tools	9 outils retournés	OK
Permissions	GET /permissions/data_engineer	4 outils autorisés	OK
load_dataset	POST /call	1 067 371 lignes chargées	OK
profile_data	POST /call	Score qualité 0,954	OK
clean_data	POST /call	1 041 670 lignes nettoyées	OK
run_analysis	POST /call	CA = 20 972 594,55 £	OK
generate_chart	POST /call	Graphique HTML généré	OK
Rejet de permission	Reporter → generate_chart	Erreur 403 Forbidden	OK

TABLE 4.5 – Résultats des tests de validation du serveur MCP

Le test de rejet de permission, en particulier, confirme que le mécanisme de contrôle d'accès du serveur MCP fonctionne correctement : une tentative d'appel de l'outil `generate_chart` par l'agent `reporter` — qui n'y est pas autorisé selon la table des permissions — est systématiquement rejetée avec le code d'erreur HTTP 403.

4.5 Artefacts produits par un run complet

Chaque exécution du pipeline (*run*) produit un ensemble structuré de fichiers, organisés dans un répertoire dédié, garantissant la traçabilité complète des résultats :

```

1 runs/run_001/
2   metadata.json          <- informations generales du run
3   tool_calls.jsonl       <- journal de tous les appels MCP
4   decisions.jsonl        <- journal des decisions de l'
   orchestrateur
5   artifacts/
6     clean.csv            <- 1 041 670 lignes nettoyees
7     insights.json        <- KPIs calcules et alertes
8     charts/
9       ca_mensuel.html    <- graphique CA par mois
10      top_pays.html       <- graphique top pays
11     dashboard.html      <- dashboard complet interactif
12     report.html          <- rapport final compile

```

Listing 4.1 – Structure des artefacts produits par un run

Chapitre 5

Interface et Dashboard

5.1 Présentation de l'interface développée

Afin de permettre à un utilisateur de soumettre un jeu de données, de suivre l'activité des agents en temps réel et de consulter les résultats produits, une interface web a été développée. Celle-ci s'articule autour de trois zones principales : une zone de soumission du jeu de données et de l'objectif d'analyse, une zone de suivi en temps réel de l'activité des agents (journal d'exécution), et une zone de consultation du dashboard et du rapport final une fois le pipeline terminé.

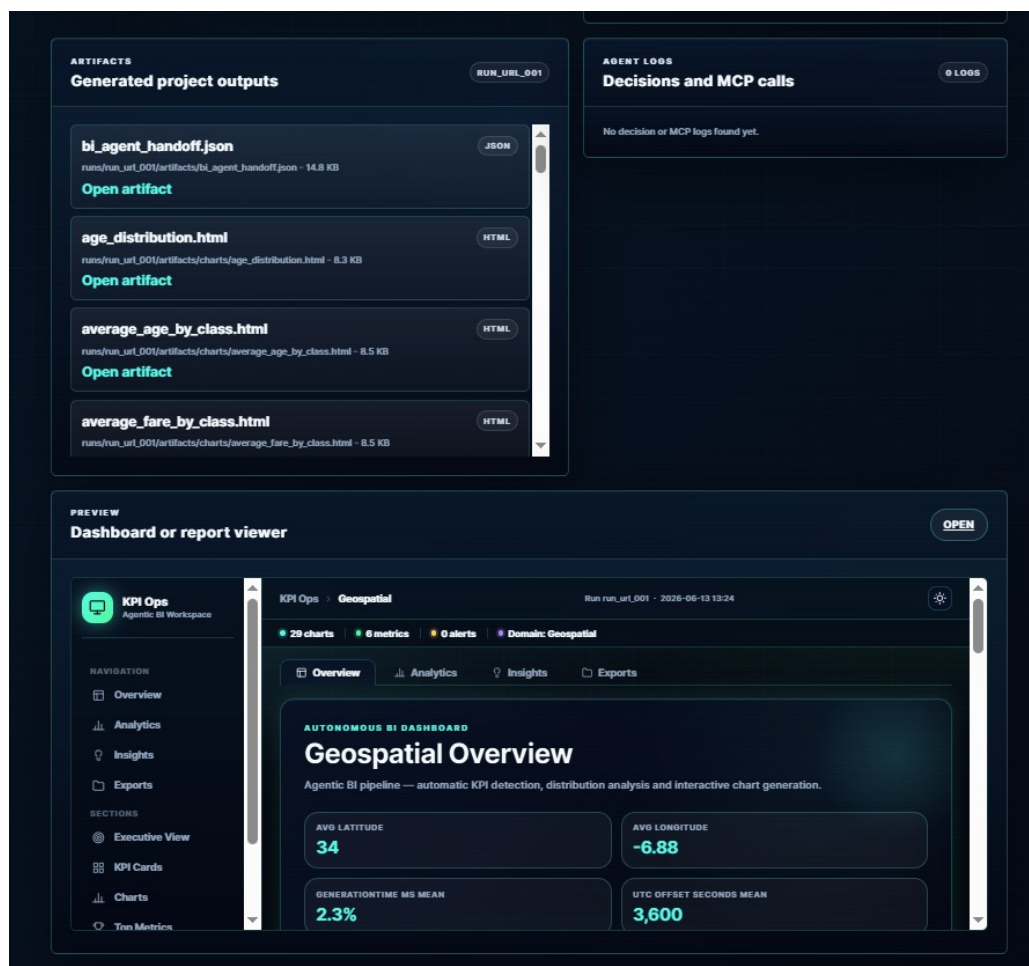


FIGURE 5.1 – Interface principale de soumission du jeu de données

5.2 Suivi en temps réel de l'activité des agents

Pendant l'exécution du pipeline, l'interface affiche en continu les actions effectuées par chaque agent : l'outil appelé, l'agent à l'origine de l'appel, le statut de l'opération (succès ou échec), ainsi qu'un horodatage. Cette vue est directement alimentée par le journal `tool_calls.jsonl` produit par le serveur MCP, garantissant que l'utilisateur dispose d'une visibilité complète sur le déroulement du pipeline.

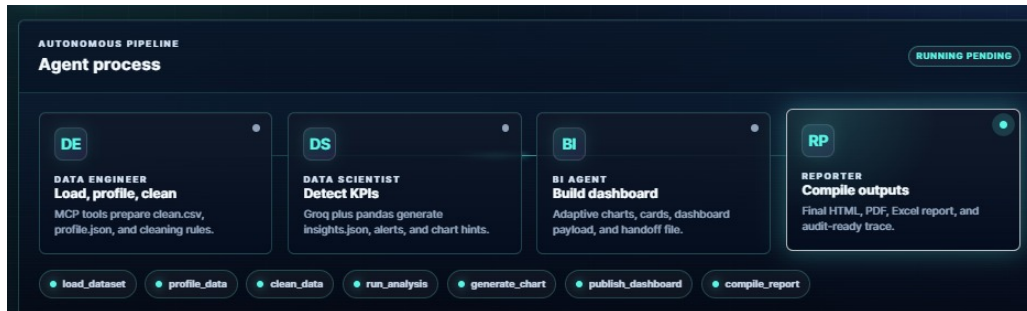


FIGURE 5.2 – Suivi en temps réel de l'activité des agents

5.3 Dashboard interactif de visualisation

Une fois le pipeline terminé, le dashboard généré par le BI Agent est accessible directement depuis l'interface. Il regroupe l'ensemble des graphiques produits à partir des `chart_hints` fournis par le Data Scientist, ainsi qu'un résumé des KPIs clés sous forme de tuiles synthétiques (chiffre d'affaires total, nombre de clients uniques, panier moyen, score de qualité des données).

0.48

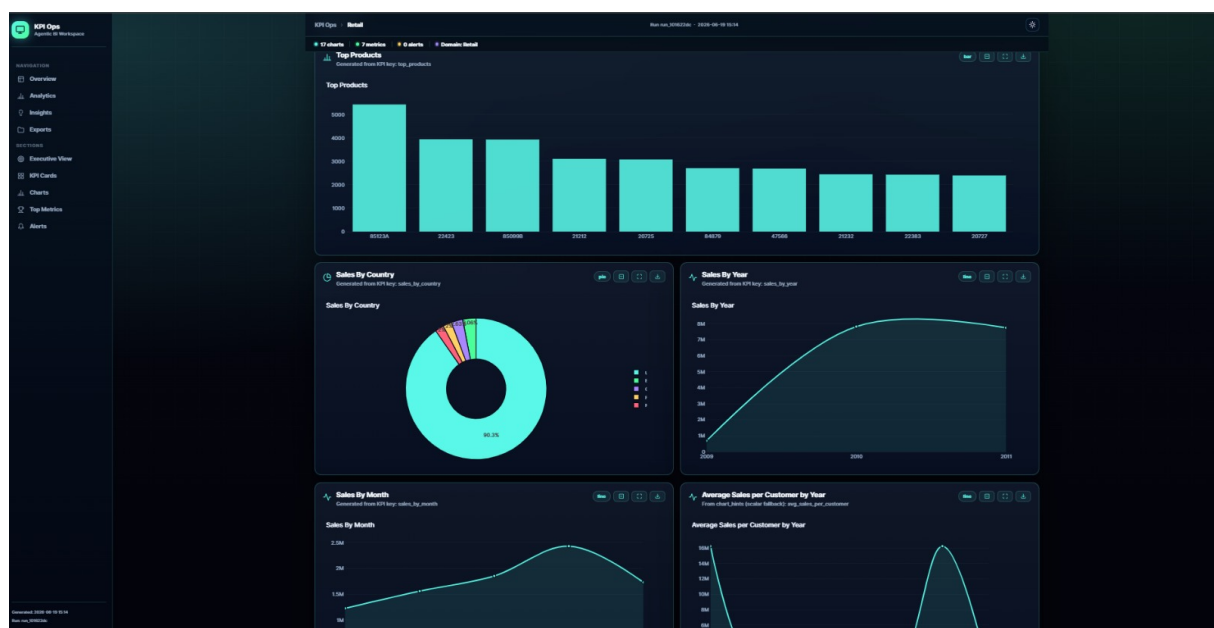
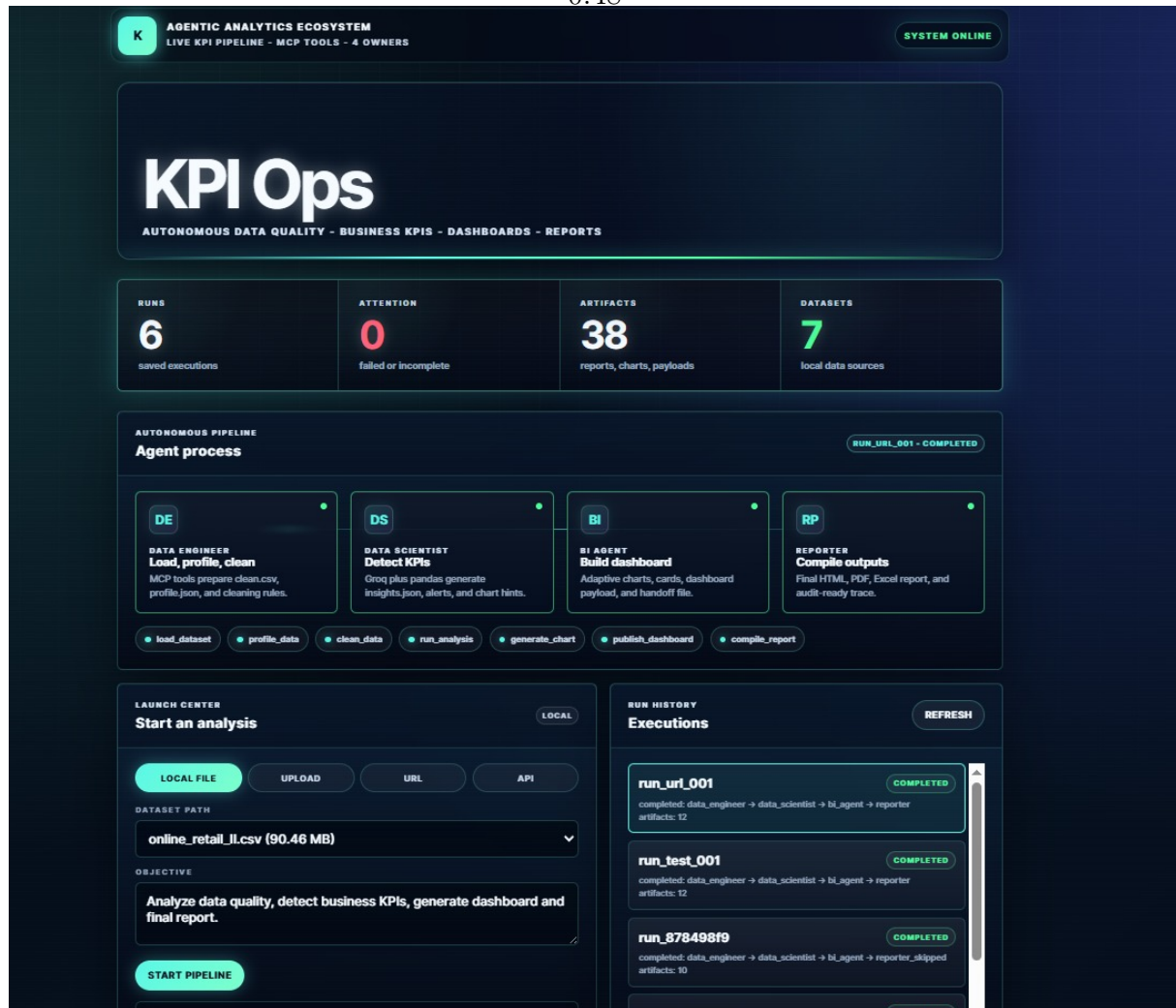


FIGURE 5.3 — Tuiles de KPIs synthétiques du dashboard

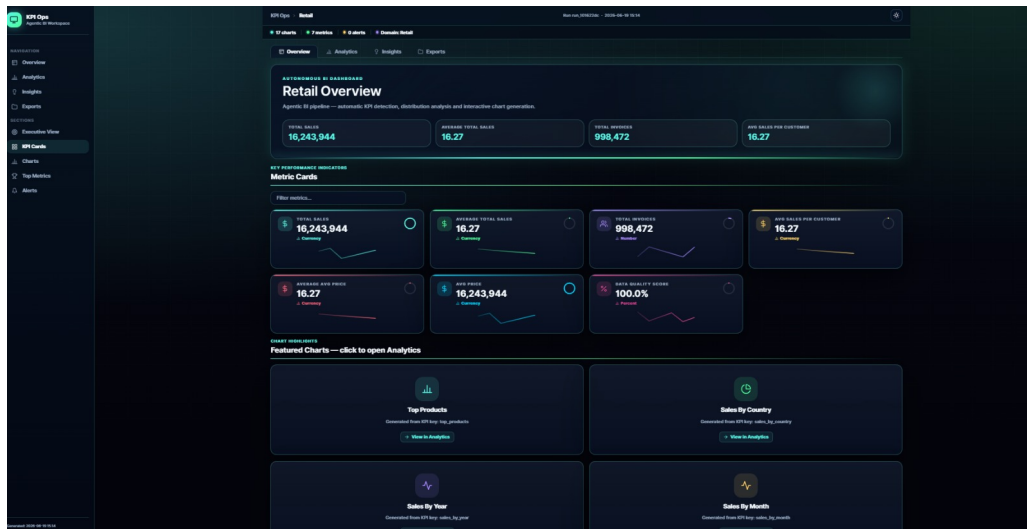


FIGURE 5.4 – Graphiques interactifs du dashboard généré par le BI Agent

5.4 Consultation du rapport final

Le rapport final, compilé par le Reporter, est également accessible depuis l'interface. Il synthétise la méthodologie suivie, les principaux résultats obtenus, ainsi qu'une référence directe vers le dashboard publié, permettant à un utilisateur non technique de consulter rapidement les conclusions de l'analyse sans devoir naviguer dans l'ensemble des artefacts produits.

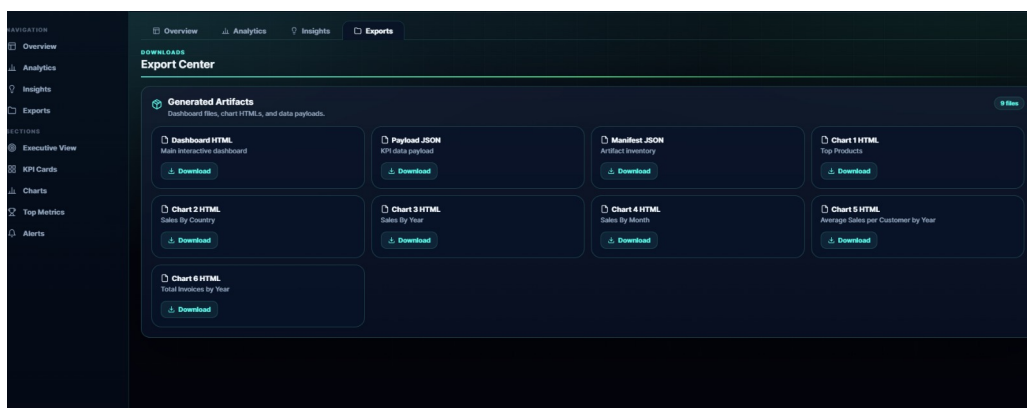


FIGURE 5.5 – rapport final

5.5 Bilan de l'interface

L'interface développée permet de rendre accessible à un utilisateur non technique l'ensemble du pipeline multi-agent, depuis la soumission du jeu de données jusqu'à la consultation des résultats, sans nécessiter de connaissance du fonctionnement interne du serveur MCP ni des agents. Elle constitue ainsi la couche de présentation du système, complémentaire à l'architecture backend décrite dans le chapitre précédent.

Chapitre 6

Démonstration et Conclusion

6.1 Démonstration de l'application

Cette section illustre le déroulement complet d'une exécution du pipeline, depuis la soumission du jeu de données brut jusqu'à la consultation du dashboard final.

6.1.1 Lancement du pipeline

```
=====
PIPELINE TERMINE – statut : completed
Taches terminees : ['data_engineer', 'data_scientist', 'bi_agent', 'reporter']
=====

=== RESUME FINAL ===
Status      : completed
Completes   : ['data_engineer', 'data_scientist', 'bi_agent', 'reporter']
Artifacts   : {'clean_csv': 'runs/run_001/artifacts/clean.csv', 'quality_score': '0.854', 'final_rows': '998472', 'insights_json': 'runs/run_001/artifacts/insights.json', 'nb_kpis': '10', 'domain': 'E-commerce', 'CA_total': 'N/A', 'dashboard_path': 'runs/run_001/artifacts/dashboard.html', 'nb_charts': '6', 'handoff_path': 'runs\\run_001\\artifacts\\bi_agent_handoff.json', 'report_path': 'runs\\run_001\\artifacts\\report.html', 'summary': "Rapport final genere avec 10 KPI(s), 0 alerte(s) et 38 chart(s). Dashboard associe: runs/run_001/artifacts/dashboard.html. PDF: generated_with_basi
```

FIGURE 6.1 – Démonstration du système MCP et du pipeline

6.1.2 Exécution séquentielle des agents

Lors de l'exécution, chaque agent est invoqué successivement par l'orchestrateur. Le journal d'exécution ci-dessous illustre le déroulement typique observé lors de nos tests :


```

=====
DATA ENGINEER AGENT - run_001
Tous les appels passent par MCP Server
=====

[DataEngineer] Etape 1/3 - load_dataset
[DataEngineer] load_dataset OK - 1067371 lignes

[DataEngineer] Etape 2/3 - profile_data
[DataEngineer] profile_data OK - score qualite : 0.854

[DataEngineer] Etape 3/3 - clean_data
[DataEngineer] clean_data OK - 998472 lignes propres

```

```

DATA SCIENTIST AGENT - run_001
LLM tool-calling via MCP Server
=====
[DataScientist] Fichier : runs/run_001/artifacts/clean.csv
[DataScientist] Colonnes anonymes : 0/11 (0%)
[DataScientist] Colonnes nommées - pas de résolution nécessaire.
  ✓ [data_scientist] → run_analysis
[DataScientist] LLM terminé - parsing JSON...
[DataScientist] KPIs disque chargés : 10
[DataScientist] chart_hints enrichis : 8
[DataScientist] insights.json patché - 8 chart_hints, 0 alias
[DataScientist] Score qualite : 1.0
[DataScientist] Domaine : E-commerce
[DataScientist] Nb KPIs : 10
[DataScientist] KPIs calculés : Total Sales, Average Price, Total Invoices, Top Selling StockCode, Customer Count
[DataScientist] Alertes : 0

```

```

[Engine] Lancement : bi_agent
[Router] Tache 'bi_agent' → agent correspondant
[publish_dashboard] charts: 6 ['trend', 'breakdown', 'products', 'mix', 'chart_1', 'chart_2']
[Engine] 'bi_agent' termine ✓ - 6 charts
[Engine] Dashboard : runs/run_001/artifacts/dashboard.html
[Planner] Prochaine tache : reporter

[Engine] Lancement : reporter
[Router] Tache 'reporter' → agent correspondant
🚀 Groq - modèle : llama-3.1-8b-instant - agent : reporter

```

```

=====
REPORTER AGENT - run_001
Compilation du rapport final via MCP Server
=====

[Reporter] Rapport genere : runs\run_001\artifacts\report.html
=====

[Engine] 'reporter' termine ✓
[Engine] Rapport : runs\run_001\artifacts\report.html

```

FIGURE 6.2 – Extrait du journal d'exécution d'un run complet

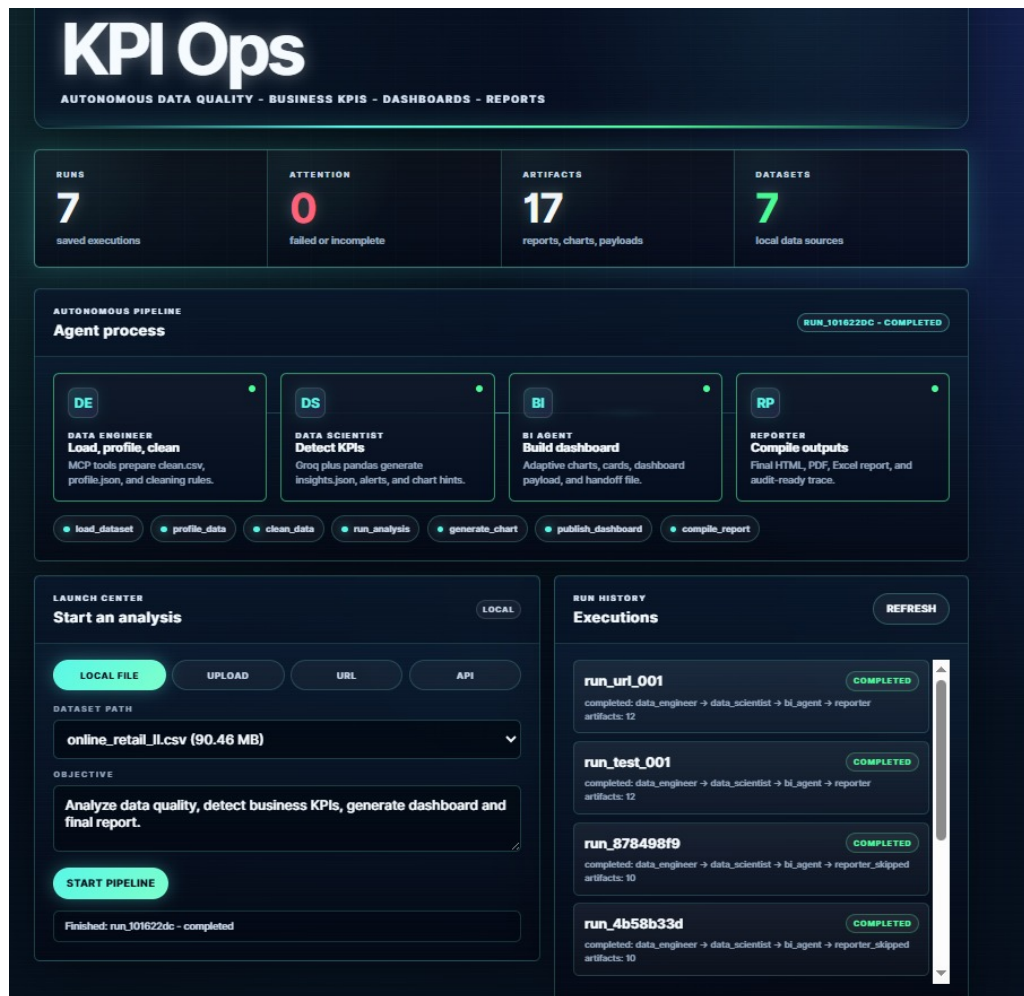


FIGURE 6.3 – Dashboard final consultable après exécution complète du pipeline

6.2 Limites identifiées

Plusieurs limites ont été identifiées au cours du développement et des tests du système :

- **Dépendance au format des données** : le système prend actuellement en charge les formats CSV, Excel et JSON ; l'ajout d'autres formats (bases de données SQL, API REST) nécessiterait l'extension de l'outil `load_dataset`.
- **Temps de traitement** : sur un jeu de données dépassant le million de lignes, certaines étapes (notamment `profile_data`) peuvent s'avérer lentes sur du matériel aux ressources limitées.
- **Dépendance à la disponibilité du LLM** : bien que des mécanismes de *fallback* heuristique soient en place, la qualité de la détection du domaine métier et de la pertinence des KPIs dépend de la disponibilité et des performances du modèle de langage.
- **Gestion des datasets totalement anonymes** : lorsque les noms de colonnes sont génériques (ex. : `col1`, `feature_2`), la détection du domaine métier devient plus incertaine et repose davantage sur les statistiques des colonnes que sur leur sémantique.

6.3 Perspectives d'évolution

Plusieurs axes d'amélioration ont été identifiés pour les développements futurs du système :

- Étendre la prise en charge à d'autres sources de données (bases SQL, API REST, flux de données en temps réel).
- Ajouter un mécanisme de résolution automatique des noms de colonnes pour les jeux de données anonymes, combinant suggestion par le LLM et heuristiques de secours fondées sur les statistiques des colonnes.
- Permettre la bascule dynamique entre plusieurs fournisseurs de modèles de langage (Groq, Claude, GPT-4o) selon la disponibilité et le budget alloué.
- Déployer le système dans un environnement conteneurisé (Docker) afin de garantir la portabilité et faciliter le déploiement en production.
- Enrichir le dashboard avec des filtres interactifs avancés (période, segment client, catégorie de produit).

6.4 Conclusion

Ce projet de fin d'année a permis de concevoir et de mettre en œuvre un écosystème d'agents IA collaboratifs, orchestrés autour d'un serveur MCP central, capable de transformer automatiquement un jeu de données brut en indicateurs de performance, graphiques interactifs et rapport d'analyse exploitable.

À partir d'un jeu de données réel comportant des problèmes de qualité authentiques, notre système a démontré sa capacité à ingérer, profiler, nettoyer et analyser plus d'un million de transactions, à détecter de manière autonome le domaine métier correspondant, et à produire des indicateurs business pertinents — un chiffre d'affaires total de 20 972 594,55 £, un score de qualité des données de 0,954, et un dashboard interactif présentant les principales tendances.

L'architecture retenue, fondée sur le protocole MCP, a permis de garantir un contrôle rigoureux des permissions de chaque agent ainsi qu'une traçabilité complète de chaque action entreprise par le système, répondant ainsi directement à la problématique de contrôle et de fiabilité posée en introduction. Le recours à un modèle de langage open-source accessible gratuitement (Llama 3.1 via Groq) a par ailleurs permis de valider l'ensemble du système sans coût d'infrastructure, tout en conservant la possibilité de migrer vers des modèles propriétaires plus performants pour un usage en production.

Au-delà des résultats techniques obtenus, ce projet illustre concrètement l'apport des architectures multi-agents pilotées par des modèles de langage pour automatiser des tâches d'analyse de données traditionnellement réalisées manuellement. Il ouvre la voie à des évolutions ambitieuses : extension à de nouvelles sources de données, généralisation à des domaines métier plus variés, et déploiement dans un environnement de production conteneurisé.

Bibliographie

- [1] Anthropic, *Model Context Protocol Specification*. Disponible sur : <https://modelcontextprotocol.io/>
- [2] Groq, *Groq API Documentation — Tool Use and Function Calling*. Disponible sur : <https://console.groq.com/docs>
- [3] Meta AI, *Llama 3.1 Model Card*. Disponible sur : <https://ai.meta.com/blog/meta-llama-3-1/>
- [4] Chen, D., *Online Retail II Dataset*, UCI Machine Learning Repository, 2019. Disponible sur : <https://archive.ics.uci.edu/dataset/502/online+retail+ii>
- [5] FastAPI, *FastAPI Documentation*. Disponible sur : <https://fastapi.tiangolo.com/>
- [6] The Pandas Development Team, *Pandas Documentation*. Disponible sur : <https://pandas.pydata.org/docs/>
- [7] Plotly Technologies Inc., *Plotly Python Open Source Graphing Library*. Disponible sur : <https://plotly.com/python/>
- [8] Wang, L. et al., *A Survey on Large Language Model based Autonomous Agents*, Frontiers of Computer Science, 2024.
- [9] Schick, T. et al., *Toolformer : Language Models Can Teach Themselves to Use Tools*, NeurIPS, 2023.
- [10] Pydantic, *Pydantic Documentation — Data Validation Using Python Type Hints*. Disponible sur : <https://docs.pydantic.dev/>